Technical Documentation

Y - Microblogging App

Concept: Development of a multiplatform social network
Application name: Y
Written by: Pablo Vaz Castrillón
Course: 2nd Year CS Multiplatform Application Development
Issue date: March 2026

Index

1. System Architecture
2. API
3. Database
4. Web Application
5. Mobile Application
6. Desktop Application
7. Security
8. Deployment
9. Usage and Maintenance
10. Future Enhancements

1. System Architecture

The developed application is a text-based social network, similar to Twitter, where users can post and view other users' posts.

The system follows a client-server architecture, where client applications consume a centralized REST API, which manages system logic and database access.

The client applications are:

- Web Application developed with Laravel.
- Mobile Application developed in Android Studio using Java.
- Desktop Application developed with Windows Forms in C#.

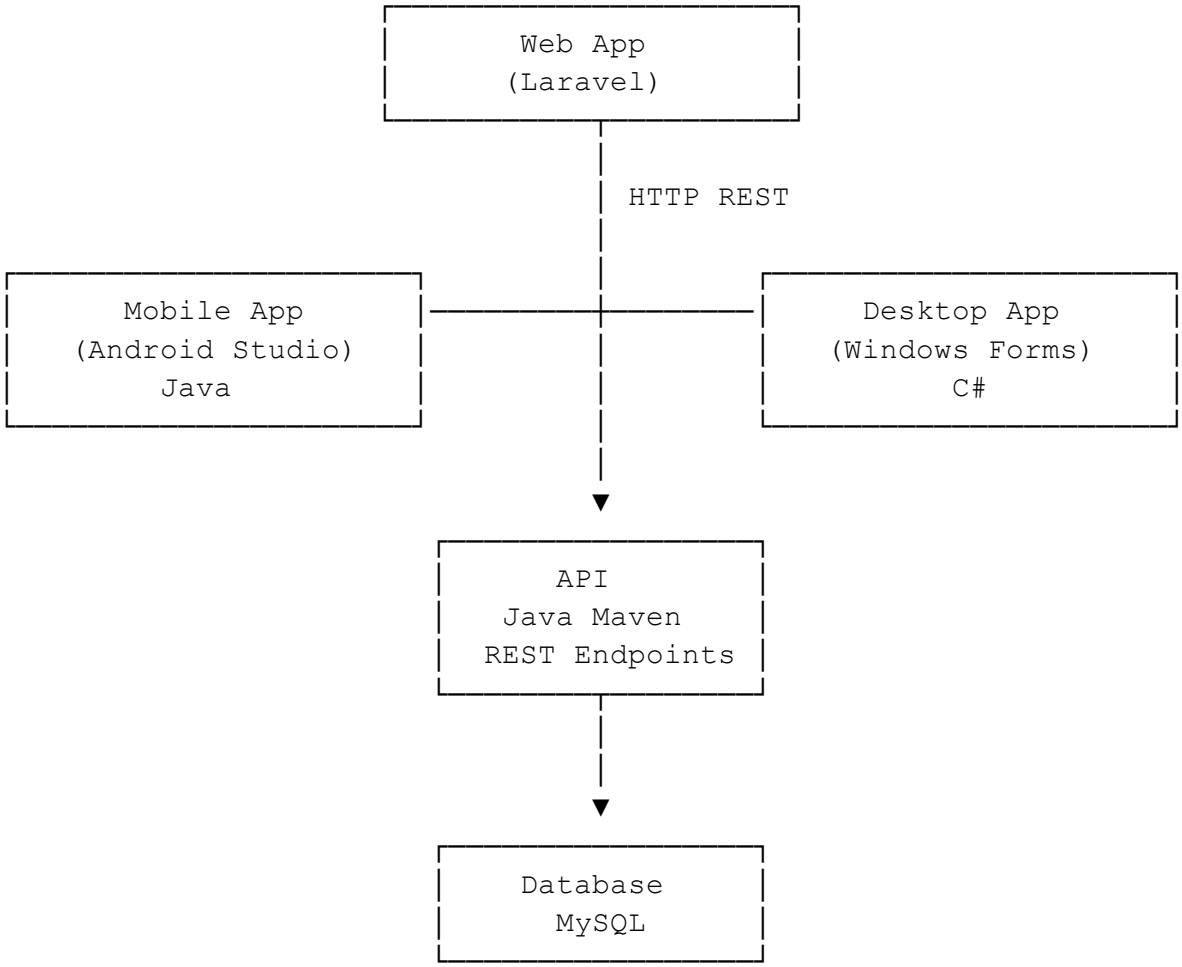
All applications make HTTP requests to the API to perform operations such as authentication, posting content, or managing users and followers.

1.1 Technologies Used

Component	Technology
API	Apache Tomcat, Java, Apache Maven
Web Application	Laravel, HTML5, CSS3 and JavaScript
Mobile Application	Android Studio, Java
Desktop Application	.NET Windows Forms, C#
Database	MariaDB/MySQL (XAMPP local environment)

1.2 System Architecture Diagram

The diagram shows how the three client applications communicate via HTTP requests with the REST API. This API acts as an intermediary between the clients and the database, centralizing business logic and ensuring data consistency.



2. API

The REST API is the core of the system, centralizing system logic and managing requests from the different client applications.

It has been developed in Java using Apache Maven for project management and deployed on an Apache Tomcat server.

Client applications communicate with the API via HTTP requests, exchanging data in JSON format.

2.1 API Architecture

The API follows a RESTful, resource-based architecture, where each system entity (users, posts, followers, etc.) represents a set of endpoints.

Each resource is implemented as a Java class containing its corresponding endpoints. For example, the Users class implements all operations related to user management.

Each method in the class corresponds to a REST endpoint handling a specific operation using the HTTP methods (GET, POST, PUT, DELETE).

2.2 Endpoints

Users

Method	Endpoint	Description
POST	/users/register	Register a new user
POST	/users/login	Login
GET	/users/{id}	Retrieve user data
PUT	/users/update/{id}	Update user data
DELETE	/users/delete/{id}	Delete a user

Posts

Method	Endpoint	Description
GET	/posts	Retrieve all posts
GET	/posts/{id_user}	Retrieve a user’s posts
GET	/posts/following/{id_user}	Retrieve posts from followed users
POST	/posts/create	Create a new post
DELETE	/posts/delete/{id}	Delete a post

Notifications

Method	Endpoint	Description
GET	/notifications/{id_user}	Retrieve a user's notifications

Followers

Method	Endpoint	Description
POST	/followers/{id_follower}/follow/{id_following}	Follow a user
DELETE	/followers/{id_follower}/unfollow/{id_following}	Unfollow a user
GET	/followers/isFollowing/{id_follower}/{id_following}	Verify if following
GET	/followers/{id}/followersCount	Retrieve the number of followers
GET	/followers/{id}/followingCount	Retrieve the number of followed users

2.3 User Authentication

User authentication is performed by sending credentials (email and password) to the login endpoint.
Currently, the system validates credentials and returns the authenticated user's information.

Note: Session tokens (e.g., JWT) are not used, so each request requires prior authentication from the client.

2.3 Request Examples

Login

Authenticate a user using email and password.

Endpoint

```
POST /users/login
```

Request

```
{
 "email": "usuario@email.com",
 "password": "123456"
}
```

Internal Process

1. The system searches the user by the provided email.
2. If the user does not exist, a 404 error is returned.
3. If the user exists, the password is compared against the stored BCrypt hash.
4. If verification succeeds, the user data is returned in JSON format.

Successful Response

```
{
 "id": 5,
 "username": "usuario1",
 "email": "usuario@email.com",
 "bio": "Desarrollador",
 "created_at": "2026-02-12"
}
```

Possible Errors

Code	Description
404	User not found
401	Wrong password
500	Internal server error

User Register

Create a new account in the application.

Endpoint

```
POST /users/register
```

Request

```
{
 "username": "usuario1",
 "email": "usuario@email.com",
 "password": "123456"
}
```

Internal Process

1. Checks if the username or email is already in use.
2. The password is hashed using BCrypt.
3. The new user is inserted into the database.

Successful Response

```
201 Created
```

Possible errors

Code	Description
409	Email already in use
409	Username already in use
500	Internal server error

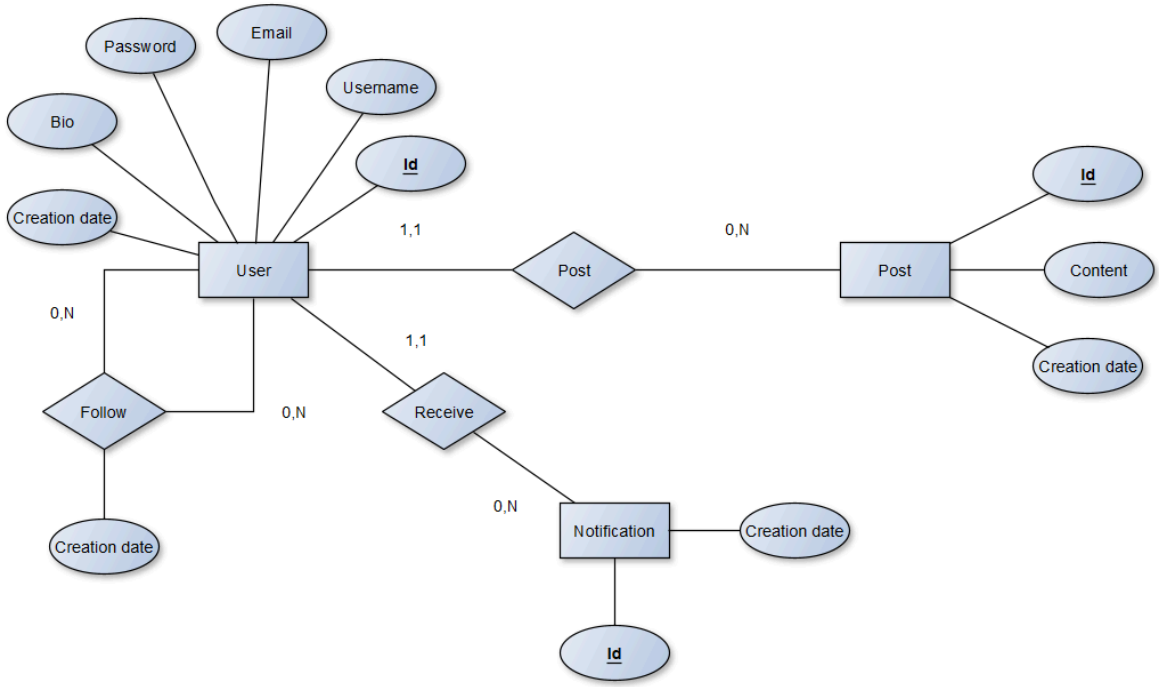
3. Database

The application uses a relational database implemented in MariaDB (compatible with MySQL), deployed via XAMPP to store information about users, posts, follower relationships, and notifications.

Database access is performed from the Java-developed API via JDBC, using Prepared Statements to enhance security and prevent SQL Injection attacks.

The database is structured into multiple tables related through primary and foreign keys, ensuring referential integrity.

3.1 Entity-Relationship Diagram



The database consists of the following tables:

- **Users**: stores user information.
- **Posts**: stores post information.
- **Followers**: stores information about follower relationships between users.
- **Notifications**: stores notifications, which are records of follower relationships.

3.2 Tables and Fields

Users Table

Field	Type	Key	Description
id	BIGINT UNSIGNED	Primary Key	Unique user identifier
username	VARCHAR(255)	UNIQUE	Username
email	VARCHAR(255)	UNIQUE	User’s email
password	VARCHAR(255)	—	BCrypt hashed password
bio	TEXT	—	User biography
created_at	TIMESTAMP	—	Account creation date

Posts Table

Field	Type	Key	Description
id	BIGINT UNSIGNED	Primary Key	Post unique identifier
id_user	BIGINT UNSIGNED	Foreign Key	Id of the user who created the post
content	TEXT	—	Post content
created_at	TIMESTAMP	—	Post creation date

Foreign Keys

Field	Reference
id_user	users.id

Behaviour

On a user deletion all its posts get deleted too (`ON DELETE CASCADE`).

Followers Table

Field	Type	Key	Description
id	BIGINT UNSIGNED	Primary Key	Relationship identifier
id_follower	BIGINT UNSIGNED	Foreign Key	Id of the user who follows another user
id_following	BIGINT UNSIGNED	Foreign Key	Id of the followed user
created_at	TIMESTAMP	—	Relationship creation date

Constraints

Composite unique key: (id_follower, id_following) that prevents duplicate follows.

Foreign Keys

Field	Reference
id_follower	users.id
id_following	users.id

Behaviour

Deleting a user automatically removes all related follow relationships.

Notifications Table

Field	Type	Key	Description
id	BIGINT UNSIGNED	Primary Key	Notification identifier
id_user	BIGINT UNSIGNED	Foreign Key	Id of the user receiving the notification
id_follower	BIGINT UNSIGNED	Foreign Key	User who triggered the notification
created_at	TIMESTAMP	—	Notification creation date

Foreign Keys

Field	Reference
id_user	users.id
id_follower	users.id

Behaviour

Stores a record of follower relationships; when a relationship is created in the followers table, a notification is also generated.

3.3 Important System Rules

- A user can create multiple posts.
- A user can follow many users.
- A user can have multiple followers.
- Users cannot follow themselves.
- Duplicate follow relationships are prevented by the composite unique key in the followers table.
- If a user is deleted, the following are also automatically deleted:

    - Their posts
    - Their follow relations
    - Their notifications

3.4 Migration Management

Database structure management is handled via Laravel migrations, while the Java API accesses data via JDBC without modifying the structure. This provides centralized and reproducible control over the database, ensuring consistency across environments (local, testing, and production).

Laravel allows rolling back migrations using the command:

```
php artisan migrate:rollback
```

This facilitates undoing structural changes.

Database Creation and Update

1. Migration files are generated in Laravel using the command:

   ```
   php artisan make:migration migration_name
   ```

2. Each file defines the creation or modification of a table, including:

   - Columns and data types
   - Primary and foreign keys
   - Unique constraints
   - Default values
   - Relationships between tables (e.g., ON DELETE CASCADE on foreign keys)

3. Migrations are executed with the command:

   ```
   php artisan migrate
   ```

This automatically applies all defined changes to the database.

Benefits of using migrations

- **Database version control:** each migration is logged, making it easy to revert changes or track schema evolution.
- **Reproducibility:** any environment can build the entire database by running migrations, without needing manual SQL.
- **Easy maintenance:** to modify the database structure (adding new tables, columns, or indexes), new migrations are created, avoiding direct database editing.

If you choose not to use Laravel

Database creation statement:

```
CREATE DATABASE `y_db` /*!40100 DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci */
```

Tables creation statements:

```
CREATE TABLE `users` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `username` varchar(255) NOT NULL,
  `email` varchar(255) NOT NULL,
  `password` varchar(255) NOT NULL,
  `bio` text DEFAULT NULL,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`id`),
  UNIQUE KEY `users_username_unique` (`username`),
  UNIQUE KEY `users_email_unique` (`email`)
) ENGINE=InnoDB AUTO_INCREMENT=31 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci

CREATE TABLE `posts` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `id_user` bigint(20) unsigned NOT NULL,
  `content` text NOT NULL,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`id`),
  KEY `posts_id_user_foreign` (`id_user`),
  CONSTRAINT `posts_id_user_foreign` FOREIGN KEY (`id_user`) REFERENCES `users` (`id`) ON DELETE CASCADE
) ENGINE=InnoDB AUTO_INCREMENT=40 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
```

```
CREATE TABLE `followers` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `id_follower` bigint(20) unsigned NOT NULL,
  `id_following` bigint(20) unsigned NOT NULL,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`id`),
             UNIQUE       KEY       `followers_id_follower_id_following_unique`
(`id_follower`,`id_following`),
  KEY `followers_id_following_foreign` (`id_following`),
     CONSTRAINT  `followers_id_follower_foreign`  FOREIGN  KEY  (`id_follower`)
REFERENCES `users` (`id`) ON DELETE CASCADE,
    CONSTRAINT  `followers_id_following_foreign`  FOREIGN  KEY  (`id_following`)
REFERENCES `users` (`id`) ON DELETE CASCADE
)       ENGINE=InnoDB       AUTO_INCREMENT=42       DEFAULT       CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci

CREATE TABLE `notifications` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `id_user` bigint(20) unsigned NOT NULL,
  `id_follower` bigint(20) unsigned NOT NULL,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`id`),
  KEY `notifications_id_user_foreign` (`id_user`),
  KEY `notifications_id_follower_foreign` (`id_follower`),
   CONSTRAINT  `notifications_id_follower_foreign`  FOREIGN  KEY  (`id_follower`)
REFERENCES `users` (`id`) ON DELETE CASCADE,
  CONSTRAINT `notifications_id_user_foreign` FOREIGN KEY (`id_user`) REFERENCES
`users` (`id`) ON DELETE CASCADE
)       ENGINE=InnoDB       AUTO_INCREMENT=41       DEFAULT       CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci
```

Integration with the Java API

Although the REST API is developed in Java using JDBC, the database structure is maintained through Laravel migrations. The API only performs read and write operations, without altering the table structure.

4. Web Application

4.1 Technology Used

The web application was developed using the Laravel framework, a PHP-based web framework implementing the MVC (Model-View-Controller) architecture pattern, which allows for a clear separation of responsibilities.

Development was performed using Visual Studio Code, an editor optimized for PHP and modern frameworks.

Main technologies used:

- Laravel as the primary framework
- PHP for backend logic
- HTML5 and CSS3 for structure and styling
- JavaScript for client-side interaction

Laravel provides tools for managing routes, controllers, views, and data access, enabling scalable, maintainable, and well-structured applications

4.2 Project Structure

The web application follows the MVC architecture, separating the app into three main components:

Component	Functionality
Model	Data management and database interaction
View	User interface
Controller	Handles logic between model and view

Due to the application's simplicity and the fact that data comes directly from an external API, Laravel models were not used.

Instead, controllers make API calls and manage data as arrays, which are then passed to views. This simplifies the architecture and avoids duplicating the data access logic already implemented in the API.

Directory structure adapted to Laravel standards:

```
.
├ app/
│   ├ Http/
│   │  └ Controllers/
│   │
│   └ Services/
│      └ ApiService.php
│
├ database/
│   └ migrations/
│
├ public/
│
├ resources/
│   └ views/
│      └ layouts/
│          └ app.blade.php
│
└ routes/
    └ web.php
```

Key directories and files:

app directory

It contains the main logic of the application.

Http/Controllers

It includes the controllers responsible for managing HTTP requests, processing user requests, and returning the corresponding responses.

Services

It contains service classes that encapsulate application-specific logic.

- **ApiService.php**
  Handles REST API communication, centralizing HTTP requests and avoiding code duplication.

database directory

It contains the files related to the management and structure of the database.

migrations

It includes database migrations, where the structure of the tables and their modifications throughout development are defined.

public directory

Contains public resources accessible from the browser, such as static files, images or scripts.

resources directory

Includes application presentation resources.

views

Contains the Blade templates used to generate the application's HTML pages.

layouts

Defines the base layouts or templates of the app.

- **app.blade.php**
  Main layout shared across different views

routes Directory

Defines application routes, which establish which controller runs for each URL.

- **web.php**
  File where the application's web routes are defined.

4.3 API Connection

The web application communicates with the REST API using HTTP requests. These requests allow the application to perform operations such as:

- User registration
- Login
- Post creation
- Follower management

Requests are sent from ApiService.php to the Java API deployed on Apache Tomcat.

Communication uses JSON, sending necessary data in the request body and receiving responses from the API.

Controllers process the API responses, converting them into data structures passed to the views.

This approach decouples the presentation layer from business logic, facilitating maintenance and allowing multiple clients to reuse the API.

5. Mobile Application

5.1 Technology Used

The mobile application has been developed using Android Studio, the official development environment for Android applications.

The programming language used for the logical part is Java, widely used in the development of Android applications.

The application allows users to interact with the social network from mobile devices, providing functionalities such as:

- Registration and login
- Viewing a post feed
- Creating and deleting posts
- Following other users
- Viewing following notifications

5.2 Project Structure

The application follows the typical Android project structure, organized into different components:

```
app/src/main/
├ java/
│   └ com.example.y/
│          ├ adapters/
│          │
│          ├ data/
│          │   ├ models/
│          │   └ Api.java
│          └ ui/
│            ├ feed/
│            ├ login/
│            ├ notifications/
│            ├ otherprofile/
│            ├ post/
│            ├ profile/
│            └ MainActivity.java
└ res/
    ├ drawable/
    │
    ├ layout/
    │
    ├ menu/
    │  └ bottom_menu.xml
    │
    ├ navigation/
    │  └ main_graph.xml
    │
    └ values/
       ├ colors.xml
       ├ strings.xml
       └ themes.xml
```

Below is a description of the directories, what they contain and their functionality briefly explained.

Data Layer

data

Manages application data access.

- **models**
  Contains classes that represent the data models used in the application (users, posts, notifications) and a singleton class (Session) to store the session user.

- **Api.java**
  Class responsible for communication with the external API, including making requests and receiving data.

Interface Layer

adapters

Includes the adapters used by RecyclerView and ViewPager, responsible for binding data to views.

ui

Contains the UI components (Activities and Fragments) organized in separate folders for each part of the application.

- **feed**
  Fragment in charge of displaying the posts available in the feed.

- **login**
  Login and register activities.

- **notifications**
  Fragment in charge of displaying current user notifications.

- **otherprofile**
  Fragment used to load the profile of other users.

- **post**
  Activity intended for the creation of new posts.

- **profile**
  Fragment showing the profile of the current user.

- **MainActivity.java**
  Main activity that manages navigation between fragments after login.

Resources

Contains the application resources.

drawable

Graphic resources such as icons, images and shapes.

layout

XML files that define the structure of user interfaces.

menu

Define the application menus.

- bottom_menu.xml: menu used in bottom navigation bar.
- menu_logout.xml: single-item menu used to end the current session.

navigation

Navigation configuration files.

- main_graph.xml: main navigation graph.

values

Reusable value files.

- colors.xml: color definitions.

- strings.xml: text strings for the UI.

- themes.xml: visual theme configuration.

5.3 API Connection

The mobile application interacts with the API via HTTP requests (`GET`, `POST`, `DELETE`), sending and receiving JSON data.

API requests are made asynchronously to avoid blocking the main thread, ensuring a smooth user experience.

API responses are processed and transformed into model objects, which are then used to update the UI.

Classes responsible for API communication handle operations such as:

- User authentication
- Fetching posts
- Creating posts
- Managing followers
- Retrieving notifications

6. Desktop Application

6.1 Technology Used

The desktop application has been developed using Visual Studio 2022 as the IDE.

The programming language is C#, along with the .NET framework and Windows Forms for creating graphical user interfaces.

This is a native desktop application for Windows, allowing users to interact with the social network without a web browser.

Functionalities include:

- User registration and login
- Viewing a feed of posts
- Creating and deleting posts
- Follow other users
- Viewing following notifications

6.2 Project Structure

The desktop application follows a modular architecture based on separation of responsibilities, which allows clear, maintainable, and scalable code organization.

```
.
├ Controllers/
│
├ Forms/
│   └ Controls/
│
├ Models/
│
├ Resources/
│
├ Services/
│  ├ ApiService.cs
│  └ Session.cs
│
└ Program.cs
```

Below is a description of the directories, what they contain and their functionality briefly explained.

Controllers

Includes the classes responsible for handling requests and responses to the REST API.

Its main function is to act as an abstraction layer between the application and the API, handling data serialization to JSON and deserialization of the received responses.

Forms

Windows Forms representing the graphical interface.

Each form manages user interaction and displays information received from controllers.

Controls

Custom reusable controls used across different forms, ensuring a consistent interface and avoiding code duplication.

Models

Classes representing system entities (users, posts, notifications…). Used for mapping API responses and structuring data internally.

Resources

Stores graphical resources such as icons, images, and visual elements.

Services

Encapsulates data access logic and application state management.

ApiService.cs

Centralizes API communication, handling HTTP requests and JSON response processing.

This approach avoids code duplication and facilitates the maintenance of the application.

Session.cs

Manages the authenticated user session, storing relevant information such as user ID and details during runtime.

Program.cs

Main entry point of the application, responsible for launching the initial form.

6.3 API Connection

The desktop application communicates with the API via HTTP requests performed in controllers and through calls from ApiService.cs, sending and receiving JSON data.

API requests are made asynchronously to avoid blocking the main thread, ensuring a smooth user experience.

API responses are processed and transformed into model objects, which are then used to update the UI.

API communication classes handle operations such as:

- User authentication
- Fetching posts
- Creating posts
- Managing followers
- Retrieving notifications

7. Security

Security is a fundamental aspect in the development of applications that manage user information.

In this project, several security mechanisms have been implemented to protect data, ensure information integrity, and prevent unauthorized access.

The main measures include password hashing, SQL injection prevention, data validation, and database integrity control.

7.1 Password Hashing

User passwords are not stored in plain text in the database. Instead, the jBCrypt library is used, which implements the bcrypt algorithm to generate a secure password hash.

The bcrypt algorithm includes an automatic salting mechanism, increasing resistance against dictionary and brute-force attacks.

During user registration, the password is hashed before being stored in the database:

```
String hashedPassword = BCrypt.hashpw(req.getPassword(), BCrypt.gensalt());
```

During login, the password provided by the user is compared with the stored hash using:

```
BCrypt.checkpw(password, hashedPassword);
```

This mechanism ensures that even if the database is compromised, user passwords cannot be easily recovered.

7.2 SQL Injection Prevention

To prevent SQL Injection attacks, the API uses Prepared Statements when interacting with the database.

Prepared statements separate the SQL query structure from user input data, preventing user input from being interpreted as part of the query.

Prepared statement example:

```
PreparedStatement ps = conexion.prepareStatement(
   "SELECT * FROM users WHERE email=?"
);
ps.setString(1, req.getEmail());
```

This approach prevents malicious users from injecting SQL code into input fields.

7.3 Database Data Integrity and Consistency

The database enforces several constraints to maintain data integrity:

Unique Keys

The `users` table includes UNIQUE constraints to prevent duplicates in key fields such as:

- Username
- Email

This ensures each user has a unique identity within the system.

Foreign Keys

Relationships between tables are enforced using foreign keys, ensuring that related records are valid. For example:

- Posts must belong to an existing user
- Follower relationships can only exist between valid users

Cascade Operations

Some relationships use:

```
ON DELETE CASCADE, ON UPDATE CASCADE
```

This ensures that when a user is deleted or updated, all related records are automatically updated or removed, such as:

- Posts
- Follower relationships
- Notifications

This prevents orphan or inconsistent records in the database.

7.4 Data Validation

The API validates data before storing it in the database. For example:

- Checks if an email is already registered
- Verifies that the username is not already in use

These checks prevent inconsistencies and system errors.

Additionally, input format validation is applied (e.g., email format or field length) to prevent invalid or malicious data.

7.5 Session Management

User session management is handled on the client side, storing authenticated user information after a successful login.

In the mobile and desktop applications, a Session class is used to maintain user state during execution.

Currently, the system does not use authentication tokens (such as JWT), so authentication relies on client-side session management.

7.6 Applied good practices

During development, several security good practices were applied:

- Secure password storage using bcrypt.
- Use of prepared statements to prevent SQL Injection.
- Input data validation.
- Database constraints to ensure data integrity.
- Separation of business logic through a centralized REST API.
- Session management in client applications.

These measures improve the overall security of the system and help protect user information from unauthorized access.

8. Deployment

The deployment of the project covers the REST API and the three client applications (web, mobile, and desktop), which work as consumers of that API.

Below are the requirements, installation steps, and execution process for each component, as well as considerations for proper operation across different environments.

For the application to work properly, the database is necessary; it can be created using the migrations of the web app's Laravel project, or by executing the creation statements listed in section 3.4.

8.1 API Deployment

The REST API was developed in Java with Maven and deployed on an Apache Tomcat server. It can run both locally and on a remote server.

Prerequisites

- Java JDK 17 or higher installed
- Maven installed and configured
- MariaDB/MySQL running on the defined port (3306 by default)
- Database created
- Tomcat server (version 9 or higher)

Deployment Steps

1. Clone the API project from the repository.
2. Build the project using Maven:

   ```
   mvn clean package
   ```

3. Copy the generated .war file from the target/ folder to the webapps/ folder in Tomcat.
   - The `.war` file contains the packaged application ready for deployment.

4. Start Tomcat:

```
catalina.bat start      # Windows

./catalina.sh start     # Linux / macOS
```

5. Access the API at:

```
http://localhost:8080/Yapi/rest
(where Yapi corresponds to the project name deployed in Tomcat)
```

6. Verify correct operation using the test endpoint:

```
GET /users/ping

If the response is 'pong', the API is working correctly.
```

8.2 Web Application Deployment

The web application is developed with Laravel and requires a local or remote server that supports PHP.

Laravel allows configuration of different environments (development, production) through the .env file, making it easy to adapt the application to different execution contexts.

Prerequisites

- PHP 8 or higher
- Composer
- Web server (Apache or Nginx)
- MariaDB/MySQL database with migrations applied

Deployment Steps

1. Clone the web project from the repository.
2. Install dependencies with Composer:

```
composer install
```

3. Configure the .env file with database credentials and API URL:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=y_db
DB_USERNAME=root
DB_PASSWORD=
API_URL=http://localhost:8080/Yapi/rest
```

4. Run migrations to create the database tables:

```
php artisan migrate
```

5. Start the server:

```
php artisan serve
```

6. Access the application in a browser:

```
http://localhost:8000
```

The web application operates as a client of the REST API.

8.3 Mobile Application Deployment

The mobile application was developed using Android Studio using Java.

Prerequisites

- Android Studio installed
- Android emulator or physical device
- REST API running and accessible from the network

Deployment Steps

1. Open the project in Android Studio.
2. Configure the API base URL in `Api.java` if necessary.

3. Connect an Android device or start an emulator.

4. Run the application (`Run > Run 'app'`).

The application will connect to the API to authenticate users, display the feed, and manage social interactions.

When using an Android emulator, replace localhost with 10.0.2.2 to access the host machine's API.

Ensure the device or emulator has network access to the API.

8.4 Desktop Application Deployment

The desktop application runs as a native Windows application and requires access to the REST API.

Prerequisites

- Visual Studio 2022 or higher
- .NET Framework 4.8 or higher
- REST API running and accessible

Deployment Steps

1. Open the solution in Visual Studio.
2. Configure the API base URL in ApiService.cs to ensure proper backend communication.
3. Build the project (`Build > Build Solution`).
4. Run the application (`Debug > Start Debugging or Ctrl + F5`).

The desktop application will function as an API client, allowing users to view the feed, create posts, follow users, and receive notifications.

8.5 Execution Environments

The system can run in different environments:

- **Development Environment:** Used during project implementation, with all services running locally (API, database, and client applications).

- **Testing Environment:** Used to validate correct system behavior before final deployment.

- **Production Environment:** Intended for end users, where the API is deployed on a publicly accessible server and client applications are configured to consume it.

Environment separation allows testing without affecting the production system and facilitates maintenance and future evolution of the project.

9. User and Maintenance Guide

This section provides detailed instructions for running the project in local environments, as well as recommendations for extending functionality and resolving common issues.

It is intended for developers and advanced users who need to maintain or extend the system.

9.1 Local Execution

To run the complete system locally, it is necessary to execute the four main components: REST API, web application, mobile application, and desktop application.

REST API (Java)

Prerequisites

- Java JDK 17 or higher
- Maven
- MariaDB/MySQL running with the database created
- Apache Tomcat 9+

Steps

1. Clone the API repository.
2. Build the project using Maven:

```
mvn clean package
```

3. Copy the generated .war file from target/ to the Tomcat webapps/ folder.
4. Start Tomcat:

```
catalina.bat start       # Windows
./catalina.sh start      # Linux/macOS
```

5. Verify the API using a test endpoint:

```
GET http://localhost:8080/Yapi/rest/users/ping
```

   It should return pong if the API is working correctly.

Web Application

Prerequisites

- PHP 8+
- Composer
- Web Server (Apache/Nginx)
- MariaDB/MySQL database with migrations applied

Steps

1. Clone the web project.
2. Install dependencies:

```
composer install
```

3. Configure .env with database credentials and API URL:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=y_db
```

```
DB_USERNAME=root
DB_PASSWORD=
API_URL=http://localhost:8080/Yapi/rest
```

4. Run migrations:

```
php artisan migrate
```

5. Start the development server:

```
php artisan serve
```

6. Access in a browser:

```
http://localhost:8000
```

Mobile Application

Prerequisites

- Android Studio
- Emulator or physical device
- REST API accessible from the network

Steps

1. Open the project in Android Studio.
2. Configure the API base URL in `Api.java` if necessary.

   For emulators, use `10.0.2.2` instead of `localhost`.

3. Connect a device or start an emulator.
4. Run the application (`Run > Run 'app'`).

Desktop Application

Prerequisites

- Visual Studio 2022+
- .NET Framework 4.8+
- REST API accessible

Steps

1. Open the solution in Visual Studio.
2. Configure the API URL in `ApiService.cs`.
3. Build the project (Build > Build Solution).
4. Run the application (`Debug > Start Debugging o Ctrl + F5`).

9.2 Adding New Features

To extend the application, it is recommended to follow these guidelines:

1. **API**
    - Create new endpoints following the existing controller structure.
    - Test endpoints using tools such as Postman or Rester.
    - Ensure compliance with security rules (bcrypt, prepared statements, data validation).
2. **Web**
    - Create controllers, views, and routes following the MVC pattern.
    - Connect new features with the corresponding API endpoints.
3. **Mobile**
    - Add new Activities and models based on the desired functionality.
    - Update `Api.java` to consume new endpoints.
4. **Desktop**
    - Create new forms and models.
    - Update `ApiService.cs` to properly consume the API

Maintaining consistency across all clients is essential to ensure new features are available on all platforms.

9.3 Common Issues

Problem	Solution
API not responding	Check that Tomcat is running and port 8080 is available
Database connection error	Verify .env (web) or JDBC configuration (API)
User already registered	Ensure email and username are unique before registration
Mobile app not loading data	Check that the API URL is accessible from the device/emulator
Desktop build error	Ensure correct .NET version and API accessibility

10. Future Improvements

Although the application meets the main project objectives, there are several opportunities for improvement and new features that could be implemented in future versions to enhance usability, performance, and user experience.

10.1 Planned Improvements

1. **Optimización del rendimiento**
    - Cache frequently requested data to reduce repeated API calls.
2. **Real-time Notifications**
    - Use WebSockets or Firebase Cloud Messaging to deliver instant notifications across all platforms
3. **Endpoints**
    - Adaptation to strict REST standards

10.2 New Features

1. **Reactions and Replies**
    - Implement “likes” or emoji-based reactions to increase user interaction.
    - Add comments on posts with support for nested replies.
2. **Private Messaging**
    - Internal messaging system for direct conversations between users.
3. **Moderation and Reporting**
    - Tools to report inappropriate content or spam.
4. **Search System**
    - Search functionality for users and posts.

5. **Followers/Following Lists**
    - Ability to view lists of followers and followed users from a profile.

6. **Private Profiles**
    - Option to make profiles private.
    - Follow request system for private accounts.

7. **Multi-language Support**
    - Support for additional languages beyond Spanish and English.

10.3 Possible Optimizations

- **Advanced Security**
    - Implement JWT tokens for API authentication.
    - Add rate limiting to prevent abuse.
- **User Experience (UX)**
    - Improve UI design following modern standards
    - Optimize navigation
- **Scalability**
    - Prepare the API and database to support a larger number of users and concurrent posts

These improvements and features represent a roadmap for the system's evolution and can be implemented incrementally without affecting the stability of the current application.